



DYNAVis: Dynamically Synthesized UI Widgets for Visualization Editing

Priyan Vaithilingam
pvaithilingam@g.harvard.edu
Harvard University
Boston, USA

Elena L. Glassman
glassman@seas.harvard.edu
Harvard University
Boston, USA

Jeevana Priya Inala
jinala@microsoft.com
Microsoft
Redmond, USA

Chenglong Wang
chenwang@microsoft.com
Microsoft
Redmond, USA

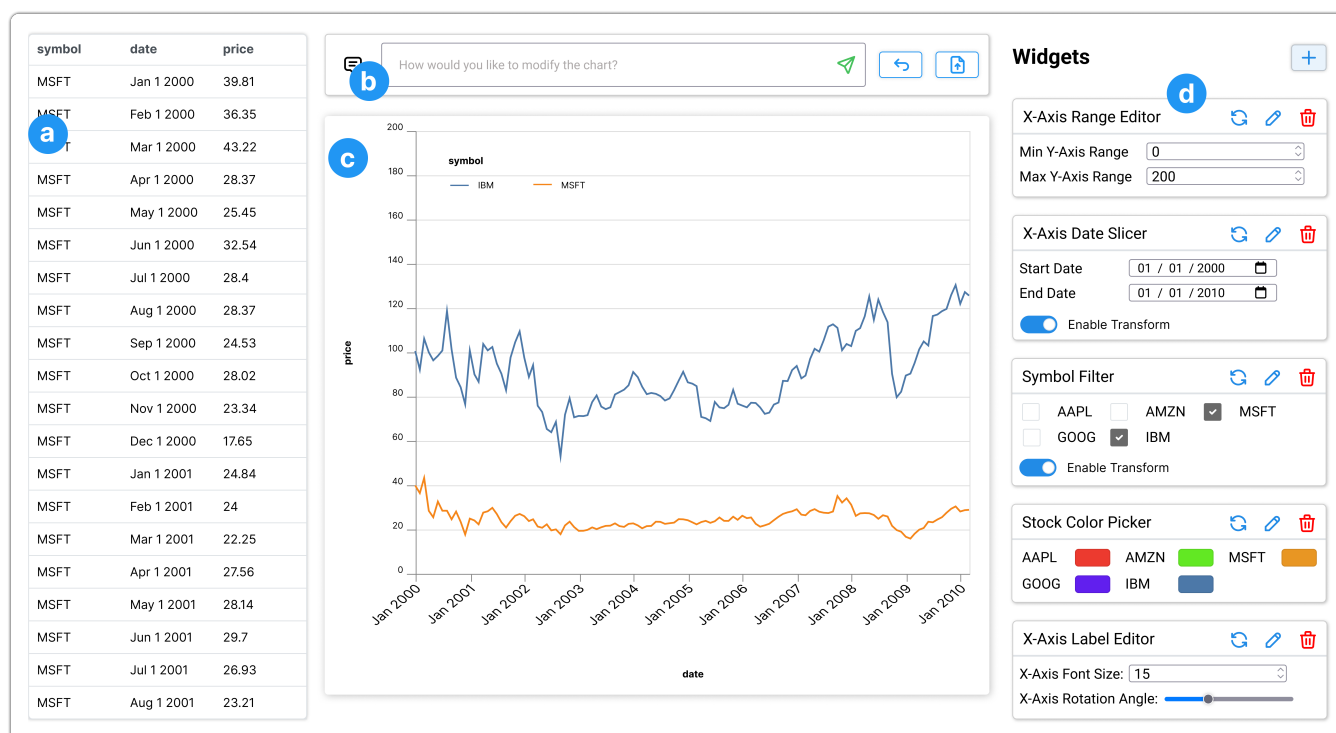


Figure 1: Screenshot of DYNAVis tool. (a) Imported data is shown on the left as a table. (b) Users can provide natural language command to edit the chart using the command bar. (c) The visualization is displayed on the center. (d) The widgets panel shows the automatically synthesized dynamic widgets based on user's natural language commands, in reverse chronological order (recently added widgets at the top).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CHI '24, May 11–16, 2024, Honolulu, HI, USA

© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0330-0/24/05

<https://doi.org/10.1145/3613904.3642639>

ABSTRACT

Users often rely on GUIs to edit and interact with visualizations — a daunting task due to the large space of editing options. As a result, users are either overwhelmed by a complex UI or constrained by a custom UI with a tailored, fixed subset of options with limited editing flexibility. Natural Language Interfaces (NLIs) are emerging as a feasible alternative for users to specify edits. However, NLIs forgo the advantages of traditional GUI: the ability to explore and repeat edits and see instant visual feedback.

We introduce DYNAVis, which blends natural language and dynamically synthesized UI widgets. As the user describes an editing

task in natural language, DYNAVIS performs the edit and synthesizes a persistent widget that the user can interact with to make further modifications. Study participants ($n=24$) preferred DYNAVIS over the NLI-only interface citing ease of further edits and editing confidence due to immediate visual feedback.

CCS CONCEPTS

• **Human-centered computing** → **Graphical user interfaces**; *Natural language interfaces*.

KEYWORDS

User Experience Design, Visualization, Usability Study

ACM Reference Format:

Priyan Vaithilingam, Elena L. Glassman, Jeevana Priya Inala, and Chenglong Wang. 2024. DYNAVIS: Dynamically Synthesized UI Widgets for Visualization Editing. In *Proceedings of the CHI Conference on Human Factors in Computing Systems (CHI '24)*, May 11–16, 2024, Honolulu, HI, USA. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3613904.3642639>

1 INTRODUCTION

Modern interactive visualization authoring tools (e.g., Tableau [10], PowerBI [6], Lyra [46], Charticulator [45]) have greatly reduced the effort to create initial visualizations from data. With these tools, authors only need to specify high-level mappings from data fields to visual properties, and behind the scenes, these tools automatically provide “smart defaults” [47, 56] to fill in hundreds of chart parameters—hiding low-level details.

While these smart defaults are often sufficient for exploratory analysis, authors who want to refine the visualization to better communicate their insights and readers who want to customize the visualization to answer their analysis objectives often find themselves in need of *editing* these default visualizations. For example, to prevent longer labels from overlapping in a line chart, the user has to rotate the labels in the x -axis. Or, the user will have to add a filter to only include data within a given date range (see Figure 2).

These edits are often considered “small tweaks” of the visualization, but these long-tailed edits can be very challenging. First, the user needs to distinguish which options will lead to the desired editing effect (e.g., understand that they need the “tick” option as opposed to “scale” or “legend” to edit label angle), which requires expertise on low-level visualization grammar. Then, the user needs to discover the edit option in the tool which may be buried in tiers of menus and panels among all others in a tool GUI (e.g., the user needs to right-click the x -axis to open its property editor, locate the sub-panel on ticks, find the rotation option to change the label angle), which can be challenging to achieve without decent tool expertise. As a result, users are either presented with a complex UI where they are swamped with options, or a tailored interface designed to simplify navigation where they often find themselves too restricted to perform the desired customization.

An emerging approach to address this visualization editing and refinement challenge is to design *natural language interfaces* (NLIs) that allow users to describe editing effects in natural language. Then, based on the user’s instructions, the tools automatically infer necessary options and corresponding values to apply the edits. For example, the user can give the natural language command to “move

the x -axis title to the left side of the axis”, which will translate to changing the “titleAnchor” property of the “ x ” encoding to the value “start”. However, while NLIs address the discovery and navigation challenges, they forgo the benefits of GUI, especially the abilities to perform fine-grained edits, obtain immediate visual feedback from editing results, and quickly undo and reapply edits. For example, if the user wants to make the width of the strokes in the line chart thicker, they do not always have the exact size in mind, and would often try out different sizes before choosing one. Or, if the user wants to change the colors of the bar chart, they may not know the exact hex (or RGB) value to provide. Such limitations restrict NLIs’ applications in visualization editing.

To address the visualization editing and refinement challenge, we design a new interaction approach, *interaction via dynamically synthesized GUI widgets*, and develop a tool named DYNAVIS for visualization editing. Our key design insight is to blend natural language interfaces with interactive GUI widgets so that users can benefit from both NLIs’ reduction in the gulf of execution [11] and GUI’s interactivity. To perform a visualization editing task, the user starts by either describing what edits they want to perform (e.g., “rotate x -axis label 45 degrees”) or directly asking for a GUI widget that they envision to perform the edits with (e.g., “give me a slider to control x -axis label angles”); either way, DYNAVIS synthesizes a GUI widget (along with a preset value from user’s specification in the former case) using a Large Language Model (LLM) for the user to explore and perform subsequent edits.

Besides the immediate benefits of reduced navigation overhead and interactivity for exploring edit effects, users can also easily compose and coordinate multiple edits using dynamic widgets as they persist after synthesis for quick editing access. Behind the scenes, we designed a widget synthesis engine powered by a large language model that translates user inputs into an HTML implementation of the widget and a call-back function that connects the widget inputs to visualization properties. DYNAVIS is highly expressive and supports both chart design edits (e.g., adjusting tick spacing, legend position, color scheme, label and title font properties) for authors to refine visualizations, and data-related edits (e.g., generating filters, zooming controllers, and sort) for readers to interactively explore visualizations without pre-built interactive widgets. Our study with 24 participants shows that participants prefer to use DYNAVIS over NLI-only interfaces due to the ease of repeating edits, and increased confidence when editing using a GUI due to immediate visual feedback.

Our contributions are as follows:

- A new interactive approach for visualization editing, dynamic widgets, that combines NLI with GUI widgets to reduce the gulf of execution and enhance interactivity.
- A widget synthesis engine that leverages large language models to translate natural language inputs into widgets and control functions.
- A user study to evaluate how users use DYNAVIS to solve visualization editing tasks.



Figure 2: DYNAVIS dynamically synthesizes widgets based on natural language commands for visualization editing. The user can describe an edit to the visualization, and DYNAVIS modifies the visualization and synthesizes a dynamic widget which the user can use for further edits (shown as $a \rightarrow b$). Alternatively, the user can directly ask for a dynamic widget to perform edits (shown as $a \rightarrow c$).

2 USAGE SCENARIO

Alice is a consultant analyzing stock trends of technology companies using spreadsheets, and she needs to create visualizations to present her analysis results to her collaborator. Below, we describe Alice’s experience of using DYNAVIS to edit and enhance her charts. Figure 1 shows the UI of DYNAVIS, which contains four main components: (a) the data panel, (b) the command bar for specifying visualization and editing commands, (c) the visualization panel that shows the current working chart, and (d) the panel of synthesized dynamic widgets that users can use to manage widgets and edit the working chart.

Initial chart. Alice starts by importing the data “stocks.csv” into the tool, and the data shows up in the data panel (Figure 1-a). To create a chart, she provides a natural language description of the chart “create a line chart showing the stock trends” in Figure 1-b. Upon submission, DYNAVIS invokes an LLM to generate a line chart based on information from the dataset and the NL description (Figure 3). Besides creating the chart using natural language, Alice can also import the Vega-Lite visualization spec she created from other tools.

Alice is not quite satisfied with the initial visualization because (1) the legend takes too much space on the right, (2) x-axis labels are too small to read, and (3) the color scheme is not ideal, Alice decides to use DYNAVIS to refine the chart. With DYNAVIS, Alice has two options to edit charts: (1) provide a natural language instruction in the command bar (Figure 1-b) to describe the edit she wants to achieve, and (2) explicitly add a widget by clicking the “+” button at the top of the widgets panel (Figure 1-d) and providing a natural

language description of the desired widget. Either way, DYNAVIS dynamically generates widgets for Alice to perform edits.

Adjusting legend position via chart editing commands.

Alice first wants to adjust the legend position to keep the legend contained within the main chart canvas. She decides to use *chart editing commands* to describe changes she wants to apply. For this, she provides the instruction “move the legend to the left of the chart” through the natural language command bar. Based on the instruction, DYNAVIS updates the visualization spec to re-position the legend. Additionally, DYNAVIS also automatically generates a widget with a drop-down menu for various legend positions pre-populated. As shown in Figure 3-(2), with this widget, Alice experiments with multiple legend positions before finalizing her final choice of “top-left corner”, which is, in fact, a better option than “left” that Alice didn’t expect in the beginning.

Coordinated editing of text size and rotation angle of x-axis labels.

Next, Alice wants to resolve issues with x-axis labels which are currently too small to read. However, this can be a quite challenging edit: increasing the font size would most likely make labels overlap with each other; and while overlaps can be resolved by rotating label angles, too much rotation would make them less readable. Thus, Alice needs to coordinate the edits to label font size and rotation angle to find the right balance. Alice doesn’t have a clear idea on what’s the optimal combination yet, thus she starts with an exploratory command “increase the x-axis text size to 20 and rotate the labels by 60 degrees”. DYNAVIS updates the chart based on the command and presents her with a “x-axis Label Editor” widget that lets her edit the text size and angles of the x-axis labels simultaneously (see Figure 4). With this, Alice can try

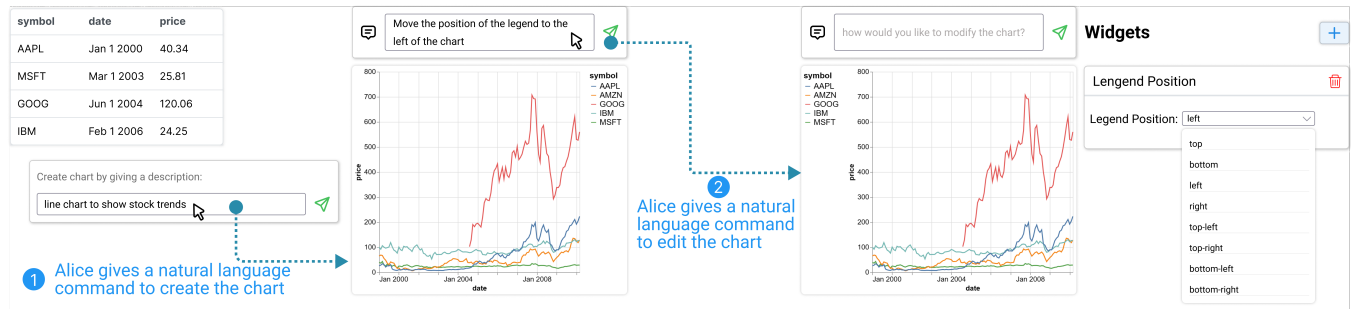


Figure 3: (1) Alice asks DYNAVIS to create a line chart to show the stock trend by providing a natural language prompt. (2) She then asks DYNAVIS to edit the legend position by giving a natural language command.

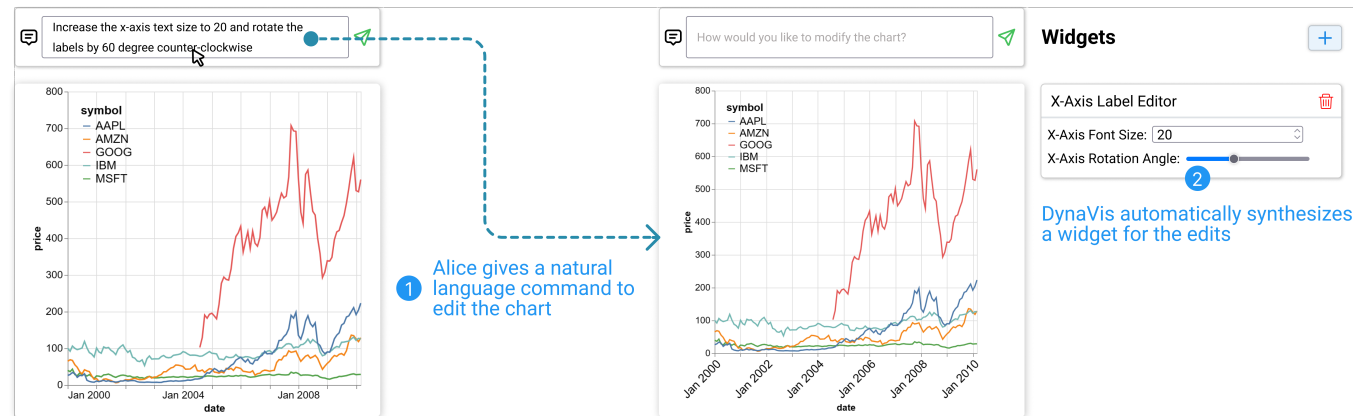


Figure 4: (1) Alice asks DYNAVIS to increase the x-axis label text size to 20 and rotate it by 45 degrees counter-clockwise. (2) DYNAVIS edits the chart and also adds a dynamically synthesized widget for Alice to make further changes in the future.

out many combinations with ease without having to re-issue the editing instructions. After some trial-and-error, she settles on a font size of 15 and rotation angle of -45 that suits her needs.

Choosing colors via widget creation commands. Now, Alice wants to modify the default color palette of the chart. Since Alice does not yet have concrete colors in mind, she decides to use the widget creation feature of DYNAVIS to ask for a widget to explore the color options. To do so, she clicks the “+” button at the top of the widget panel (Figure 5) to open the widget creation command box, she then types in the prompt “*change the color of each stock symbol*”. DYNAVIS, which understands the current context of the chart and the current dataset, generates a tailored “Stock color picker” widget that allows her to pick colors for each stock symbol in the chart (see Figure 5). DYNAVIS even knows all the symbols that are part of her current dataset, and can generate a tailored widget for her. Alice can use this widget to try different colors, get instant visual feedback, and choose the desired colors for her chart.

Data exploration with data filter widgets. After Alice finishes up the customization, she emails the visualization to her collaborator Alex who plans to include the visualization in a news article he is writing. After some analysis, Alex wants to compare the stock trends for just MSFT and IBM to get deeper insights.

Instead of going back and asking Alice to do that, Alex imports the visualization spec in DYNAVIS to perform the edits (a JSON Vega-Lite spec that contains data as elaborated more in section 3). After importing, Alex provides the command “*compare only MSFT and IBM*”, and DYNAVIS quickly updates the chart to run a filter transformation to show only the data for MSFT and IBM. DYNAVIS also provides a “SYMBOL FILTER” widget with a checkbox for each stock symbol, using which Alex can continue to compare different stock trends choosing one or more symbols to compare (Figure 1, third widget). DYNAVIS also intelligently identifies that this widget has a data-filtering transform and provides a switcher for users to enable or disable the transformation.

Next, Alex asks for a Date slicer widget to zoom the visualization by using a smaller time window (Figure 1, second widget). Since the stock prices of the two companies that are compared are much lower than other companies, Alex also requests a widget to slice the y -axis range. Now using the generated y -axis range slicer, Alex can zoom in and out of the range window to see minute price changes within the time window of his choosing (Figure 1, first widget). After finding the desired visualization, Alex includes the final visualization in his article for publication.

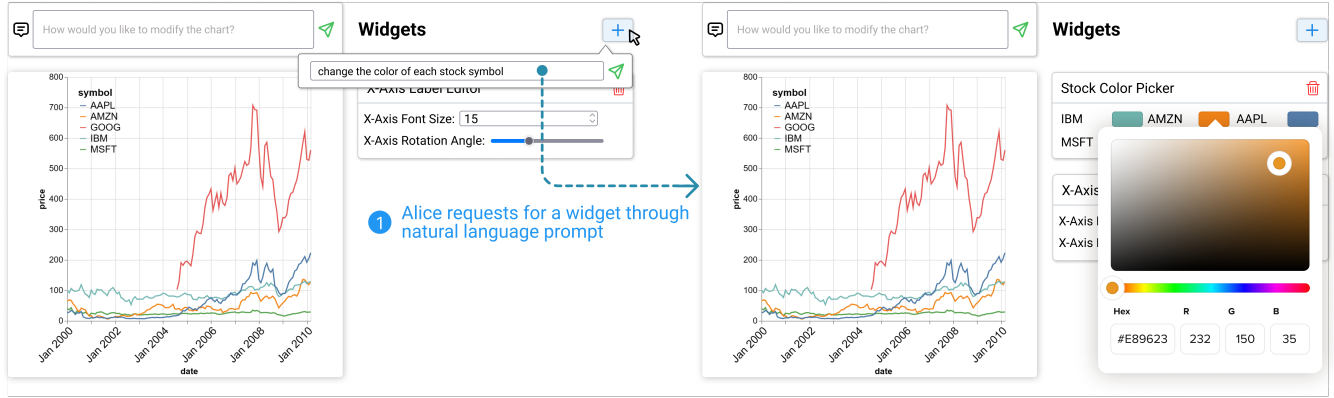


Figure 5: (1) This time Alice explicitly requests for a widget to change the color of each stock symbol by clicking the “+” button and giving the natural language prompt. (2) DYNAVIS synthesizes a widget, without making any edits to the chart.

Remark: Alice and Alex can complete the visualization refinement and exploration tasks with ease thanks to the following benefits of DYNAVIS.

- In conventional GUI, Alice needs to navigate menus and panels to locate widgets to perform edits, which requires her proficiency in both GUI and visualization terminologies. DYNAVIS lowers this barrier by allowing Alice to obtain desired widgets via natural language descriptions.
- Synthesized widgets allow Alice to perform fine-grained edits and obtain immediate visual feedback from editing results, which enables her to explore and coordinate editing options. This allows her to find optimal edits through trial and error for edits she previously didn’t know precisely (e.g., color, rotation angle, and text). Alice won’t be able to explore edit options easily with an NLI as it requires her to describe concrete parameter values and has a delayed specification-feedback cycle.
- DYNAVIS is highly expressive and supports both chart refinement edits for the author Alice and data manipulation edits for the reader Alex. Without DYNAVIS, Alex would either have to interact with Alice every time he wants an updated version of the chart or ask Alice to create an interactive visualization which requires additional efforts to tailor options.
- As dynamic widgets persist after creation and are fully compositional, Alice and Alex can go back and repeat edits (e.g., update x -axis range as analysis objective changes) or revert certain edits (e.g., undo data filtering). This can be challenging with an NLI as changes are non-compositional, and every update requires a new editing command. Or, with a conventional GUI, they would need to keep track of all edits they have done in order to repeat or redo edits.

3 DYNAVIS SYSTEM DESIGN AND IMPLEMENTATION

In this section, we first describe the design principles behind our core concept of widgets. We then describe our synthesis framework for dynamically synthesizing these widgets. DYNAVIS is a

cross-platform web application that is implemented with React and Typescript for the front-end UI and Python for the back-end server.

3.1 Dynamic Widgets

Widgets as modular sub-components. With DYNAVIS, we introduce *dynamic widgets*, which are small modular UI components that focus on a particular edit or interaction task at hand. The edit can involve simple changes to one or multiple chart properties, such as changing the position of the legend or slicing, or can involve a data transformation operation executed on the data before chart rendering, such as filtering specific ranges on the axes. At a high level, a widget has two components:

- An *HTML script* that describes the UI elements of the widget which the user will interact with to manipulate the visualization. e.g., An HTML script containing an `<input>` element of type slider to change the x -axis label size of the chart (see Figure 6).
- A *JavaScript callback function* that contains the code that will be executed to manipulate the visualization and/or data whenever the user interacts with the widget. This callback function is of the form `callback(event, chart) => (transforms, chart)`, which accepts both the HTML event object and the current chart as inputs and generates an optional list of transforms and the updated chart as outputs. This callback function is attached to the `onChange` event handlers of all the input elements in the HTML script.

An example of the dynamic widget is shown in Figure 6 along with its HTML script and Javascript callback function.

One of the main design choices of our system is to ensure the modularity of the widgets so that edits from two different widgets do not conflict or manipulate the chart in unexpected ways. This requirement has implications for how we handle charts and transforms:

Handling charts. To ensure that each widget only changes a small component of the chart, we recognized that using a declarative chart representation is preferable to an imperative one. One such declarative representation is to use a JSON object (called a

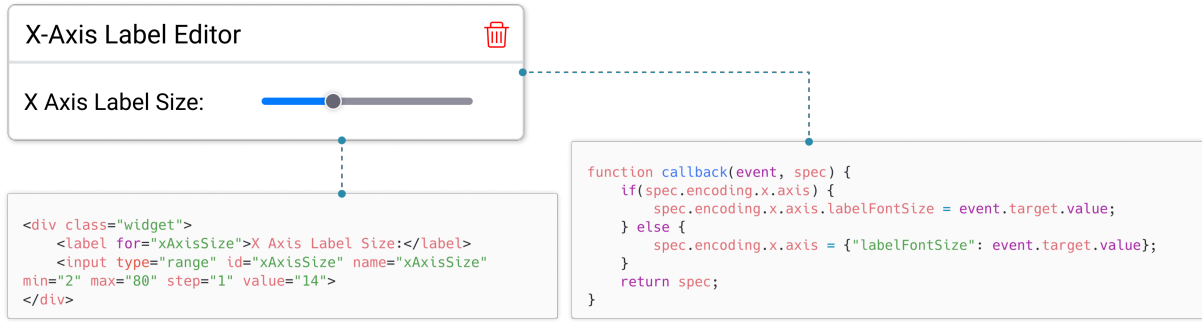


Figure 6: A dynamic widget is comprised of two components — (1) The HTML script defining the UI (2) The JS callback function that listens for the changes in the UI to edit the visualization.

specification) to encode the properties of the chart. For example, with JSON representation, to change the x-axis title, one can just edit the chart specification as `spec.encoding.x.title = "axis title"` without having to change anything else regarding the chart.

In this paper, we use Vega-Lite specifications [47] for representing charts since it provides us with a concise and declarative representation of visualization while also maintaining expressiveness. Vega-Lite also provides a well-supported rendering engine that is compatible with all the major web frameworks.

Handling data transformations. Data transformations are an important part of visualization editing. DYNAVIS handles all the Vega-Lite data transformations, e.g., filter, fold, flatten, etc. In DYNAVIS, transforms are represented as a list of objects similar to Vega-Lite. Each widget’s callback outputs a list of transforms. The transforms are performed in the order in which they are specified in this list. If there are multiple widgets with transforms, we execute transforms in the chronological order of the widgets. To enable this, we keep a mapping of the widgets to their most recent callback’s output transforms, so that we can execute all widget’s latest transforms before rendering the chart for every edit. For every widget that adds transformations to the chart, DYNAVIS adds a switch so that users can enable or disable the transform. e.g., Let’s take the example of Alex from Section 2. When they request a widget to slice the date range, the synthesized widget adds a *filter transform* on the data. DYNAVIS identifies this as a special *Transform Widget*, and allows Alex to dynamically enable or disable the transform to see the original chart and the filtered chart.

3.2 Synthesis Framework

Below we describe our synthesis framework by splitting it into three stages: pre-processing, LLM-based synthesis, and post-processing. Our framework comprises of 3 main modules: a Data Summarizer, a Chart Engine, and a Widget Engine as shown in Figure 7.

3.2.1 Pre-processing: Data Summary and Visualization Pre-processing.

To generate visualizations and synthesize dynamic widgets, the LLM needs an accurate context of the data the user is working with. However, due to the limited context window supported by LLMs, we cannot pass the whole data to the LLM. Hence, to augment LLMs with grounding context about the data, we borrow the Data

Summarizer from Lida [22]. This summarizer is used to produce a dense yet compact summary for any given dataset that is useful as a grounding context for visualization tasks. For every LLM query in the subsequent steps, we pass the data summary instead of the data. First, the summarizer applies rules to extract dataset properties including atomic types (e.g., integer, string, boolean), general statistics (min, max, unique values, etc.), and a random list of *n* samples for each column using the pandas python library [8]. Then the base summary is enriched by an LLM to include a semantic description of the dataset (e.g., a dataset of stock prices for top 5 tech companies for 10 years), and fields (e.g., Stock price in USD) as well as field semantic type prediction [62] (see Figure 8).

Before sending any edit or widget synthesis queries to the LLM, DYNAVIS splits the data from the visualization specification, and any other unnecessary chart properties (like canvas configuration properties) to prevent context overflow. This also ensures that the rendering engine can remain responsive to UI changes.

3.2.2 LLM-based Synthesis of Visualization and Dynamic Widgets.

Synthesizing visualization. The *Chart Engine* is primarily responsible for synthesizing visualizations — either new visualizations or editing existing visualizations based on the natural language prompt from the user. The Chart Engine uses the enriched data summary from the summarizer, the user prompt, and optionally an existing Vega-Lite specification to return a Vega-Lite specification using a LLM [Fig 9]. To ensure that the LLM produces a valid Vega-Lite specification, and to prevent version-based errors, we instruct the model to only use the Vega-Lite Schema v5 and to produce a valid JSON output in a markdown code-block format with language descriptors. Since chat-based models like GPT 3.5 are trained to produce verbose output, using a Markdown code-block format ensures we can reliably extract the code block. To maintain consistency of output, we provide some few-shot learning examples of Vega-Lite specification with its description to the model.

Synthesizing dynamic widgets. The *Widget Engine* is primarily responsible for synthesizing dynamic widgets. The widget engine uses the data summary, current Vega-Lite chart specification, and the user prompt to return the widget components—both HTML script and the Javascript callback function.

To ensure that the LLM produces a valid program for the widget we provide strict templates for HTML and JavaScript. The HTML

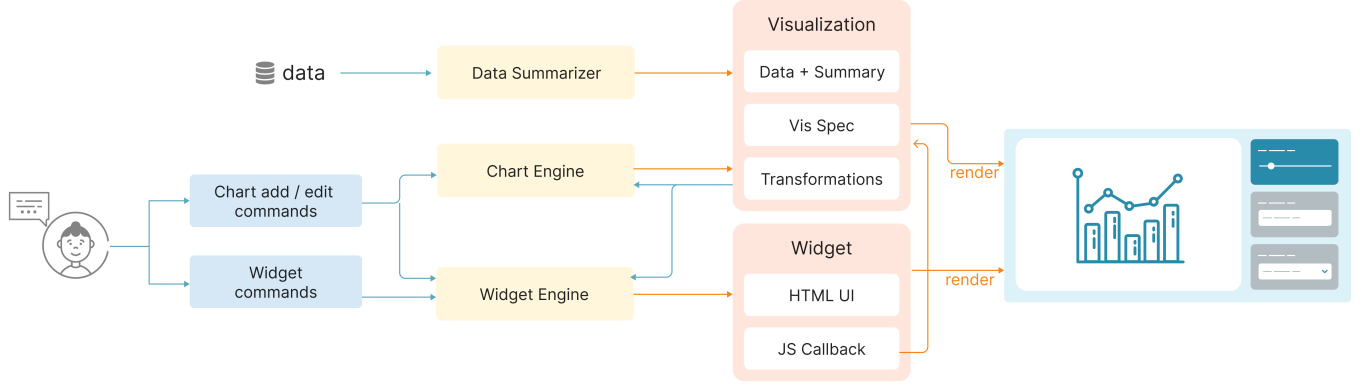


Figure 7: DYNAVIS system architecture. The Data Summarizer generates a summary from input data to assist visualization and widget synthesis. The Chart Engine applies changes to visualizations based on chart editing instructions, and the Widget Engine is responsible for synthesizing dynamic widgets from both types of user commands.

symbol	date	price
AAPL	Jan 1 2000	40.34
MSFT	Mar 1 2003	25.81
GOOG	Jun 1 2004	120.06

name: "stocks.json"
description: "A dataset containing stock prices for tech companies"
columns:
(1) name: "symbol", dtype: "category", semantic_type: "stock_symbol"
unique_values: 5, samples: ["AAPL", "MSFT", "GOOG", "IBM", "AMZN"],
(2) name: "date", dtype: "date", semantic_type: "date"....

Figure 8: The Summarizer constructs an NL summary from extracted data properties.

```
{
  "mark": "line",
  "encoding": {
    "x": {"field": "date", "type": "temporal"},
    "y": {"field": "price", "type": "quantitative"},
    "color": {"field": "symbol", "type": "nominal"}
  }
}
```

user gives a natural language command to create a chart

rotate the x-axis labels by 45 degrees counter clockwise

```
{
  "mark": "line",
  "encoding": {
    "x": {"field": "date", "type": "temporal", "axis": {"labelAngle": -45}},
    "y": {"field": "price", "type": "quantitative"},
    "color": {"field": "symbol", "type": "nominal"}
  }
}
```

Figure 9: Given the data summary, the NL prompt, and an (optional) chart specification, the chart engine produces an updated chart specification.

template defines the empty <div> stub with commented instructions. The JavaScript template has an empty JS function stub with a predefined return value. This template improves the reliability and predictability of the code generated. Templates also help us verify the code in the next step of Program Analysis. We also provided a few-shot examples of HTML and JS code to the model to maintain consistency of output.

3.2.3 Post-Processing through Program Analysis.

Processing visualizations. Once we extract the JSON specification from the markdown output, we parse the specification to check for JSON formatting errors and then compile the Vega-Lite specification to check for syntax or schema errors in the synthesized specification. In case of errors, we provide the error message and ask the model to fix the errors in the same conversation context. If this doesn't work, we re-try the prompt once again.

Processing dynamic widgets. To prevent errors, and ensure the validity of the widgets, DYNAVIS performs a series of post-processing steps on the HTML and JS code synthesized by the LLM using program analysis. We mention some of the many steps involved in the post-processing stage below. Each step is achieved by parsing the HTML and JS code to an abstract syntax tree (AST) and manipulating the AST.

- Parse the HTML code, to ensure there are no conflicts in the HTML "ID" property between the synthesized widgets and previous widgets. If we find conflicts, we programmatically modify the ID property and modify the corresponding JS callback function.
- Parse the synthesized JS callback function to ensure it has the right function name and valid function parameters.
- Identify and replace the HTML IDs used in the callback function that were modified in the HTML script.
- DYNAVIS ensures that the properties of the chart being edited are either already present in the current chart or the callback function handles the null case correctly.

3.3 User Interface Implementation

DYNAVIS is implemented as a web application with React and Typescript. We use the html-react-parser [7] library to attach and detach widgets on the fly as they are created and deleted. DYNAVIS's backed hosts the data summarizer (implemented in pandas) implemented as a flask web server that communicates with the front-end using REST API. We use the OpenAI API [1] to issue queries to the LLM. We chose gpt-3.5-turbo as the target LLM in our current implementation from OpenAI because it strikes the right balance of accuracy

vs. speed. Since we had to make edits and synthesize widgets within interactive time to prevent annoying the users, the GPT-3.5 model was responsive enough and accurate enough to suit our needs. We have also tested our tool with the more advanced GPT-4 model, which supports longer context and has better instruction-following capability to generate widgets more accurately. However, the latency is too high for smooth interaction.

4 USER STUDY DESIGN

To understand how users can use DYNAVIS to solve visualization editing tasks, we conducted a within-subjects lab study with 24 participants. In the study, users are asked to solve two sets of five visualization editing tasks, one set using DYNAVIS and another using a baseline NLI-based tool. We aim to answer the following research questions:

- RQ1** Does DYNAVIS reduce users' efforts to edit visualizations?
- RQ2** In what scenarios do users prefer to use dynamic widgets compared to a baseline tool?
- RQ3** What are users' strategies to work with dynamic widgets?

4.1 Participants

We recruited 24 participants (13 female, 10 male, 1 chose not to disclose) through the mailing lists of two research universities. Of the 24 participants, 2 worked with data visualization at least once daily, 4 participants worked with visualizations weekly, 14 participants worked with visualizations at least once a month, and 4 participants less frequently but still occasionally worked with data visualization. Participants mentioned they had prior experience with a variety of visualization tools and libraries including matplotlib (Python), Seaborn (Python), ggplot (R), Tableau, Excel, D3, and more. None of the participants had any experience with Vega-Lite or Vega libraries. Three of the 24 participants reported they performed data analysis daily, 6 participants did it weekly, 8 participants did it at least a few times a month, and 7 participants did it occasionally (less than a few times a month). Participants received a \$25 Amazon gift card as compensation for their time.

4.2 Study Conditions

We consider the following two conditions in our user study. We choose the NL-based visualization editing approach as the baseline [23].

- **Baseline Condition (UI_{NL}):** We modified the DYNAVIS interface with the support for Natural Language Commands along with a set of pre-populated UI widgets for basic chart manipulation (e.g., chart title, axis range, etc.) and remove the support for dynamic widgets. By providing both NL and basic UI supports for users to freely choose from, we believe this is a fair state-of-the-art baseline tool for the study. Pre-populated widgets are similar to static UI users use to edit visualizations (like Excel, Google Sheets, etc.). For the editing tasks, users can use a combination of natural language commands and the pre-populated widgets to edit the chart.
- **Experiment Condition (UI_{DW}):** This is DYNAVIS with the support for dynamic widgets on top of UI_{NL}. With this UI,

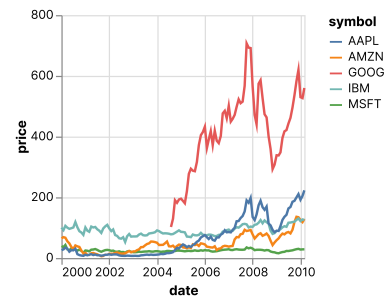
the users can edit the chart using natural language commands, add custom dynamic widgets, or use the same set of pre-populated widgets. As an addition, whenever the user provides a natural language edit command to the AI, we will synthesize and automatically add a dynamic widget. We display the synthesized widgets in reverse chronological order, so the latest synthesized widget is shown at the top.

Note that we didn't explicitly set up a widget-only baseline based on an existing visualization tool as it would require us to restrict participants to only people with experience with a certain visualization tool, hence limiting the diversity of the participants. However, we do collect user feedback in the interview on their opinions on how their experiences with DYNAVIS differ from their favorite tools.

4.3 Tasks

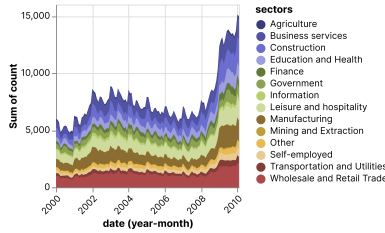
We selected three data visualization tasks derived from popular datasets. Since our focus is on visualization editing tasks, not the authoring task, the participant starts each task with a base dataset and a base visualization. Informed by the formative study conducted by Wang et al. [55], the participants will have to perform a series of edits to the visualization. Each task contains five sub-tasks containing instructions to perform edits to the base visualization. We include the following three types of questions: (1) editing the visualization with a concrete editing task, (2) exploratory editing task (e.g., try a few options and then pick the best stroke width), and (3) editing tasks with a question to be answered based on the chart (e.g., zoom in to a time window or range, filter data, etc.).

Task 1 (Stock Trends). Given a dataset of stock prices for the top five tech companies over ten years and the baseline chart below visualizing the stock trend, the user is asked to complete the following sub-tasks.



- (1) Change the chart title to "Stock Trend".
- (2) Try different lines stroke widths and find the best one that suits you.
- (3) Edit the chart to show the stock trends for AAPL and MSFT only.
- (4) Change the y-axis max range to 250 to get a zoomed in view. Then, answer the question: "By just viewing the chart, get the minimum and maximum stock price for both AAPL and MSFT".
- (5) Now compare only between IBM and MSFT. Then, answer the question: "By just viewing the chart, find the month and year, where the difference in stock price between the two companies was maximum".

Task 2 (Unemployment Data). Given a dataset of USA unemployment statistics over multiple sectors and the stacked area chart below visualizing the distribution of unemployment numbers, the user is asked to complete the following sub-tasks.



- (1) Change the x-axis title to “Timeline”.
- (2) Try out different legend positions (inside and outside the chart) to choose the best position that suits you.
- (3) Edit the chart to show the trends for Construction and Agriculture sectors only.
- (4) Edit the y-axis max date to 06/01/2004 (June 2004). Then, answer the question: “By just viewing the chart, get the approximate month and year where the difference in the unemployment rate between the two sectors were maximum”.
- (5) Now compare only between Finance and Construction. Then, answer the question: “By just viewing the chart, get the approximate month and year where the difference in the unemployment rate between the two sectors were maximum”.

Task 3 (Weather Data). Given a dataset of Seattle weather for ten years and the stacked bar chart below visualizing the aggregated distribution of weather over each month, the user is asked to complete the following sub-tasks:



- (1) Change the y-axis title to “Number of Records”.
- (2) Rotate the x-axis labels by 45, 65, and 90 degrees and choose the best one for you.
- (3) Change the color of each weather type to the following colors: Sun to Yellow, Snow to Gray, Rain to Blue, Fog to Green, Drizzle to Purple.
- (4) Edit the chart to only show “Snow” weather for all the months. Then, answer the question: “By just viewing the chart, find the month with the lowest snow days”.
- (5) Edit the chart to only show “Fog” weather for all the months. Then, answer the question: “By just viewing the chart, find the months with the highest fog days”.

4.4 Study Procedure

To enable easy access to DYNAVIS, we hosted both the control and the experiment versions of the tool online which could be accessed by participants via their web browser. After obtaining user consent, we recorded the audio and the screen-cast of each participant, and the users were encouraged to think aloud during the study. In each study session, the participant completed one of the three tasks using the control condition and another task with the experiment condition. To mitigate the learning effect, both the order of task assignment and the order of tool assignment were counterbalanced across participants through random assignment. Therefore, for each unique combination of 3 tasks and 2 conditions, we have 8 participant data points. Before each task in a study session, the participant was given a tutorial of the assigned tool and was allowed to explore using the tool for 5 minutes with a test dataset. Before starting each task, we also explained the dataset and the base visualization and provided time for the participants to explore and understand the dataset. We set a time limit of 15 minutes for each task. After each task, the participant filled out a post-task survey to reflect on their experience using the tool. After finishing both tasks, participants answered a final survey to directly compare the two conditions. At the end of each study session, we also conducted a brief informal interview at the end of the study to get the participant’s subjective experience participating in the study and feedback for the tool.

4.5 Measurements and Analysis

We recorded both quantitative and qualitative metrics during the user study. We measured the success/failure for each editing sub-task the participant had to perform. A sub-task is considered failed if the participant is unable to finish the task despite multiple attempts with the tool. They are allowed to retry as many times as they prefer. Through app telemetry, we also recorded all the natural language commands the user provided to the tool and the interactions with the widgets in the tool. In the post-task survey the user filled after every task, we recorded self-reported NASA Task Load Index, self-reported Likert scores for ease of completing the task, and how well the AI understood their intent (for survey questions look at Table 1). In the post-study survey the user filled at the end of both the tasks, we recorded the participant’s self-reported preference and modified the NASA Task Load Index that focused directly on comparing their experience between the two tools (for survey questions look at Table 2). For qualitative analysis, the first author performed open-coding on the participants’ responses, and the audio transcripts to identify themes, and then discussed with co-authors to refine the themes over multiple sessions. These themes are used to explain the qualitative results. We use paired t-test to measure the statistical significance of quantitative metrics.

5 USER STUDY RESULTS

5.1 Task Completion

Participants using the UI_{NL} tool failed to complete sub-tasks *more* often than the participants using the UI_{DW} tool. When using the UI_{NL} tool, seven participants (P4, P5, P6, P10, P11, P16, P17) failed

- Q1.1. It was easy to complete the tasks using the tool provided. (1-Strongly Disagree, 7 - Strongly Agree)
 Q1.2. The AI understood my intent and made the right edits. (1-Strongly Disagree, 7-Strongly Agree)
 Q2.1. How mentally demanding was this task with this tool? (1-Very Low, 7-Very High)
 Q2.2. How hurried or rushed were you during this task? (1-Very Low, 7-Very High)
 Q2.3. How successful would you rate yourself in accomplishing this task? (1-Perfect, 7-Failure)
 Q2.4. How hard did you have to work to accomplish your level of performance? (1-Very Low, 7-Very High)
 Q2.5. How insecure, discouraged, irritated, stressed, and annoyed were you? (1-Very Low, 7-Very High)

Table 1: After each task, participants rated (on a 7-point Likert scale) their experience (questions 1.1 - 1.2) and the subjective workload using NASA TLX measures (questions 2.1 - 2.5).

- Q1.1. Which tool would you prefer to use? (1-UI_{DW}, 7-UI_{NL})
 Q2.1. Which tool was more mentally demanding to communicate? (1-UI_{DW}, 7-UI_{NL})
 Q2.2. Which tool made you feel hurried or rushed during the task? (1-UI_{DW}, 7-UI_{NL})
 Q2.3. Which tool made you feel successful in accomplishing the task? (1-UI_{DW}, 7-UI_{NL})
 Q2.4. For which tool did you work harder to accomplish your level of performance? (1-UI_{DW}, 7-UI_{NL})
 Q2.5. Which tool made you feel more insecure, discouraged, irritated, stressed, and annoyed? (1-UI_{DW}, 7-UI_{NL})

Table 2: After finishing both the tasks, participants comparatively rated (or a 7-point Likert scale) their tool preference (question 1.1) and the subjective workload using NASA TLX (questions 2.1 - 2.5) comparing between the two conditions. (note: in the survey, the names of the UI were coded to prevent bias).

one of the sub-tasks compared to two participants (P1, P20) when using the UI_{DW} tool. None of the participants failed on more than one sub-task during the study. It is important to note, that these failures are despite participants retrying the tasks as many times as they want. Figure 10 shows the self-reported score (Likert scale; higher is better) for ease of completing the task with the tool provided for each condition. Participant using UI_{DW} found it significantly ($p = 0.004$) easier to complete the task ($\mu = 6.26, \sigma = 0.86$) compared to using the UI_{NL} ($\mu = 5.13, \sigma = 1.6$).

The average time $\bar{t}$ for participants in the UI_{NL} to complete the task is 7 minutes and 22 seconds ($\bar{t}_{\text{task}_1} = 6'05''$, $\bar{t}_{\text{task}_2} = 9'23''$, and $\bar{t}_{\text{task}_3} = 6'37''$). In the UI_{DW} condition, participants took an average time of 6 minutes and 36 seconds ($\bar{t}_{\text{task}_1} = 6'20''$, $\bar{t}_{\text{task}_2} = 7'07''$, and $\bar{t}_{\text{task}_3} = 6'23''$) to complete the task. The difference in task completion time between the two conditions is not statistically significant. Note that the exploratory nature of some sub-tasks (e.g., task 1.3) encourages participants to spend time exploring the visualization, and the completion time is not a definitive measure of performance. We report the completion time as a setup for understanding user efforts which will be discussed in the following sections.

We analyzed the session recordings to identify the root cause of these task failures. Six of the seven failures in the UI_{NL} condition and both the failures in the UI_{DW} condition happened in Task 2.4, where the participants had to slice the date range for the chart. This was due to the model returning the *dates* in an incorrect format when editing the Vega-Lite spec from an NL command. Vega-Lite uses a specific JSON date format, e.g. {"date": 14, "month": 3, "year": 2004}, whereas the model in many instances used strings to represent the date (e.g. "2004-03-14"), resulting in an error. When generating the widget, the model seems to make this mistake fewer times compared to editing the chart specification directly. The remaining failure in the UI_{NL} condition occurred during Task 1.5, where the participant had to change the filtering values from [MSFT, AAPL] to [MSFT, IBM]. In this instance, the model added an

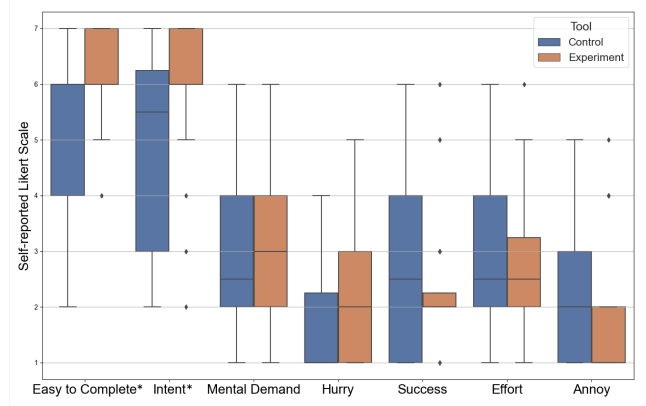


Figure 10: Participants self-reported scores for NASA TLX questions, ease of completing the task, and how well the AI understood their intent.

extra conflicting filter transformation instead of editing the existing transformation.

5.2 Self-reported Cognitive Task Load Index

In the post-task self-reported NASA TLX ratings where participants scored their cognitive load performing the tasks in both the conditions (check Table 1 for questions), we did not find any statistically significant difference in the mental demand, how hurried or frustrated they felt, the effort required to complete the tasks and their perception of success (see Figure 10).

In the post-task survey where the participants directly compared their cognitive load between the two conditions (see Table 2 for questions), when using the UI_{DW} tool, 80% of participants felt less mental demand, 83% of participants felt less hurried, 62% of participants felt more successful, 75% of participants spent less

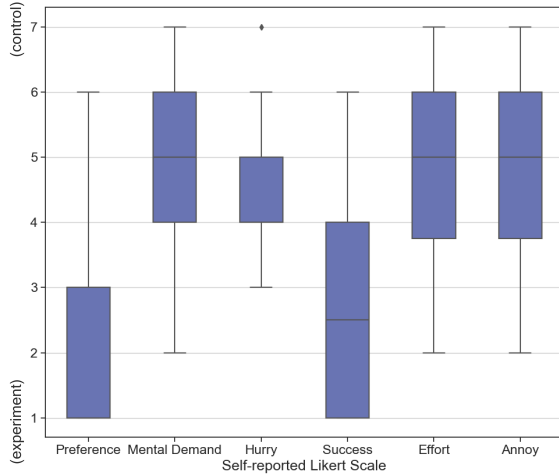


Figure 11: Participants self-reported cognitive load and preference scores that directly compare the two conditions.

effort, and 75% of the participants felt less frustrated compared to when using UI_{NL} tool (see Figure 11).

5.3 User Behavior

5.3.1 Natural Language Commands vs. Dynamic Widgets usage. Figure 12 shows the usage data for the number of natural language commands invoked by the participants, and the number of interactions with both dynamic widgets and pre-populated widgets.

In the UI_{NL} condition, participants used an average of 8.4 natural language commands per task (including all sub-tasks) to edit the visualization. This usage significantly ($p < 0.001$) decreased to just 2.67 natural language commands when using the UI_{DW} condition. This was indeed replaced by the dynamic widget usage, where participants on average used it 13.25 times when using the UI_{DW} condition. This confirms our observation of an increase in the number of times participants fine-tuned when using UI_{DW} condition. The widgets reduced the barrier to trying out many different values before settling on the edits. For instance, P6 said “With dynamic widgets, I like how easy it is to toggle or make edits in small increments and try out many different values. I can see what I’m clicking and have fine-grained controls”. There was no significant difference in the usage of pre-populated widgets between the two conditions.

5.3.2 User Strategy. In the UI_{NL} condition, we observed two major strategies used by participants to perform the tasks. Six of 24 participants (P7, P9, P12, P13, P15, P17) always used natural language commands to edit the visualization irrespective of any pre-populated widgets available to perform the edit. Whereas, the other 18 participants first looked for a pre-populated widget to perform their edits before resorting to natural language commands.

In the experiment condition UI_{DW} , we observed two major strategies used by participants. Nine participants (P2, P3, P4, P5, P6, P8, P17, P18, P21) preferred to use only the widgets to make all the edits. These participants used the AI only to synthesize dynamic widgets without making the edits and then chose to make the edits

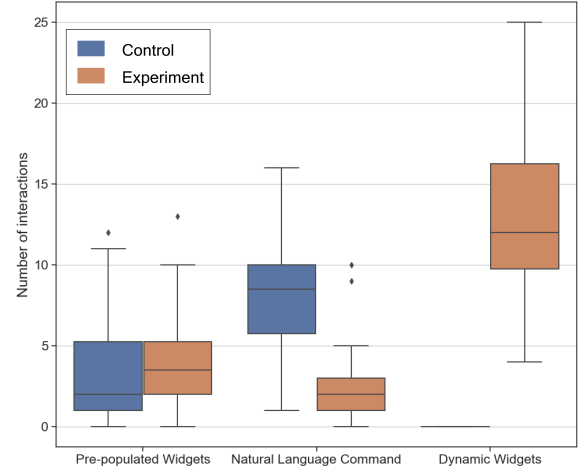


Figure 12: Usage metrics show the number of times the user used pre-populated widgets, dynamic widgets, and natural language commands.

by interacting with the synthesized widget instead. Ten participants (P7, P10, P12, P13, P16, P19, P20, P22, P23, P24) preferred to use the natural language command to perform the broad edit, and then use the automatically synthesized dynamic widgets to make fine-grained edits. P23 said “I would prefer to use the edit prompt for the first time, and use the dynamic widgets for further edits and adjustments”. P15 chose to use only natural language commands to make the edits and said “I think that [UI_{NL}] was easier to use. It is quite easy to make edits quickly using AI commands”. The remaining participants used a combination of widgets and NL commands with no particular preference.

5.3.3 Prompting Strategy. Participants when prompting the AI to edit the chart generally provided their intent (goal) to the AI. For all the sub-tasks, they structured their prompt as [ACTION VERB] + [CHART PROPERTY] + [VALUE]. Some examples are “Change the stroke width to 5”, “Rotate the x-axis labels to 65 deg”, “change the x-axis label to timeline”.

However, we did notice a variety of NL commands for filtering-based sub-tasks (Tasks 1.3, 1.5, 2.3, 2.5, 3.4, 3.5). Some participants directly mentioned the values to keep. e.g., “show me finance and construction sectors only”. Some participants mentioned values to remove. e.g., “Remove everything other than MSFT and AAPL” or “Remove the plot for AAPL. Show me the variation of IBM”. Other participants who weren’t proficient with charting libraries used incorrect chart property names like “visibility of the strokes with only MSFT and AAPL selected”, “remove all legend categories except construction and agriculture”. In three such instances, the model still performed the right filter transformation. For two instances, the model tried to modify the wrong property (like opacity) resulting in an incorrect chart.

When prompting the AI for widgets, participants skip providing the value in the prompt and directly manipulate it via the UI. Their prompt was generally structured as [ACTION VERB] + [CHART PROPERTY]. Some examples are, “change x-axis limit”, “change date

range”, “Change the x-axis max date”. For instance, P3 said “Just an action along with the part of the chart that needed to be edited”. P23 said “I keep it as short as possible. Similar to [edit] commands, but without the data”. In the next sub-section, we show evidence of how this makes prompting for a widget easier than using ML commands to directly edit the chart. By taking advantage of data context, domain constraints, and clever prompt engineering using templates within DYNAVIS, we can generate dynamic widgets with very little ambiguity with just simple instructions from users [60].

5.4 User Preference

In the post-task survey, the users were asked to rate their preference between the UI_{NL} and UI_{DW} conditions. All the participants except P15 (96%) strongly preferred using DYNAVIS compared to the baseline tool. The informal interview and survey responses shed light on the reasons, which we discuss in this sub-section.

Finding 1: Dynamic widgets make repetitive edits easier.

Sixteen participants (P1, P3, P4, P5, P6, P7, P9, P11, P13, P14, P17, P18, P19, P21, P22, P23) explicitly mentioned that dynamic widgets greatly reduced effort needed to perform repeated edits. By using dynamic widgets, they reduce the number of natural language commands they need to type – which is very time-consuming. P5 said, “I will rather just interact with the dynamic widgets. Compared to typing commands, I would just interact with an UI at the end of the day”. P7 commented “Once the widget was there for a certain type of task, I preferred using the widget as clicking is less cumbersome than articulating and typing”. Moreover, when using dynamic widgets, participants interacted with the LLM only once when creating the widgets. Subsequent edits are performed instantly when compared to using NL commands, where they have to wait for the response from LLM every time they have to perform an edit. P6 said, “Dynamic widgets were the foundation for the edits I made and I can do however many times I want. Faster than asking the AI again and again”. This also explains the increased usage of dynamic widgets mentioned in (Section 5.3.1). Users using UI_{DW}, tried a lot more values for exploratory sub-tasks since it was easier to repeat the edits, compared to using UI_{NL}.

Finding 2: Visual feedback enhances understanding and exploration. Nine participants (P2, P4, P5, P8, P11, P17, P18, P23, P24) mentioned they prefer using dynamic widgets due to the instant visual feedback the widgets provided when performing the edits. This visual feedback works both ways. First, when the user makes the edits using the dynamic widgets, the edits reflect on the chart immediately, since we aren’t waiting for a response from the LLM. P23 summarizes this by saying “The real-time update with widgets, rather than waiting for AI, was super cool. I can interactively see what happens with the chart”. It is important to note that these visualizations are inherently static with no interactivity – but by creating and using dynamic widgets, participants are interacting with an inherently static visualization.

Second, the UI provides visual feedback by retaining the edit information in the state of the UI. P24 explains this by saying “with dynamic widgets, I can also visualize the changes. For example, when I change the color, I can just see the color directly in the widget. That is

very helpful, that is just efficient”. Dynamic widgets provide clear visibility of chart (system) status, improving the visual feedback [42]. In other words, dynamic widgets help users overcome the usability challenge of the gulf of evaluation (understanding the system state) [11, 43].

Finding 3: Prompting to create dynamic widgets is easier and more reliable. Figure 10 shows the participants’ self-reported score (Likert scale; higher is better) for how well the AI understood participants’ intent. Participants using UI_{DW} tool ($\mu = 6.04$, $\sigma = 1.36$ Likert scale) reported that AI understood their intent significantly better ($p = 0.024$) than participants using UI_{NL} tool ($\mu = 4.95$, $\sigma = 1.77$ Likert scale). 18 of the 24 participants using UI_{NL} had at least one failed natural language edit command, and 8 of the 24 participants had more than 2 failed natural language edit commands. In contrast, only nine participants using UI_{DW} had at least one erroneous widget, and no participant had more than 2 erroneous widgets when performing the tasks.

Twelve participants (P1, P2, P3, P5, P6, P11, P14, P16, P17, P21, P22, P24) explicitly noted that it was easier and more reliable to prompt for dynamic widgets than to use just natural language to edit the visualization. For instance, P3 commented, “with [UI_{NL}] I had to provide my chat command as specific as possible to prevent errors. But with dynamic widgets in [UI_{DW}] I didn’t have to be very specific”. P6 commented on the reliability by saying “I like the ability to execute the edits using dynamic widget rather than leave it up to chance using the AI”.

One probable reason for improved reliability could be: when using only natural language commands, the LLM will have to generate the modified Vega-Lite spec for the whole chart in the response. This increases the probability of errors, since during the generation, the LLM can potentially change any other unrelated property in the specification. Whereas, when generating dynamic widgets, the model synthesizes a JavaScript callback function that only edits one (or a few) properties of the Vega-Lite spec, greatly reducing the probability of errors.

Finding 4: Dynamic widgets enhance the sense of control over NL. Seven participants (P3, P5, P6, P8, P10, P16, P20) mentioned that using dynamic widgets provided them a greater sense of control over the outcome when making edits to the visualization. Users felt that the widgets provided a sense of structure to the edits performed and a sense of autonomy. P8 said “The widgets gave me a feeling of autonomy to make more changes compared to the AI. It had created some kind of structure for achieving the change”. Further, P6 said “typing the text commands was a little non-ideal. Not having the widgets made it feel like I don’t control the edits. [natural language] commands made it feel like there was some level of uncertainty”.

Finding 5: Dynamic widgets enable customization, but can be overwhelming for extended use. Nine participants (P1, P3, P6, P9, P10, P11, P16, P14, P18) mentioned they liked using dynamic widgets since it allowed them to customize their own UI to suit their editing intents. P7 said “The dynamic widgets are pretty cool. It’s like writing a whole tool for yourself, but instead of writing code, it is instantly available”. Since we present the widgets in the reverse

chronological order of their creation, P18 said “*I like dynamic widgets, because it sort of stores the history of edits I made. I can refer it back whenever I want to change it if I don’t like*”.

Compared to traditional static UIs, dynamic interfaces are constantly changing. Five participants (P21, P17, P18, P8, P23) pointed out that creating dynamic widgets can get overwhelming over long editing sessions with widget panels constantly being changed, and wanted some way of either pausing widget creation or ability to categorize and control the widgets that are shown. For instance, P23 said “*If I made a lot of them [dynamic widgets], then it might get tricky. It’ll be nice to have automatic grouping based on the kind of edits the widgets do*”. Similarly, P8 said “*It would be nice to group, collapse, or create multiple relevant and related widgets, just like Photoshop, to customize better*”. In contrast, P7 said “*If it was a long series of tasks, I would maybe just prefer a stable interface*”, and P15 said — “*with [UI_{DW}] I’m not sure how dynamic widgets are helpful. Using the commands was just simpler*”.

5.5 Tool Performance

To understand the performance of the DYNAVIS using the GPT3.5 model, we performed a postmortem analysis to measure the latency faced by the participants due to the model response time, and the number of automatic retries required to generate the correct Vega-Lite specification and widget code. We measured this by re-playing all the *NL visualization edit* commands, and the *NL add widget* commands provided by the participants during the user study. On average, each query to the GPT3.5 model added a latency of 1.47 seconds ($\sigma = 0.34$ seconds). Also, on average for each NL edit / add widget command, DYNAVIS had to automatically retry 1.16 times ($\sigma = 0.58$ times) to fix syntax or semantic errors before producing the correct output.

6 DISCUSSION AND FUTURE WORK

Modalities beyond NL and UI widgets. DYNAVIS employs two different modalities for users to specify the kinds of edits they want to make to the chart: natural language commands and interaction through dynamic widgets. This design combines the strengths of both modalities so the user can better communicate the intent to the AI agent and quickly make repeated precise edits to the visualization. This was reflected in the study by the participants’ preference for the tool, and how they interacted with both modalities.

An interesting future direction is to expand the modalities beyond the natural language and widget interaction by adding support for voice commands as well as gestures and direct manipulation of the chart. For example, instead of the user giving an NL command “*Move the legend to the bottom of the chart*”, the user can simply click and drag (though mouse, touch, or digital pens) the legend to the bottom of the chart to make the edit. The user doesn’t need to know they have to modify *legend*, rather they can simply point us toward it. Similarly, DYNAVIS also has the potential to include by-example specifications to let users demonstrate editing examples by editing parts of the visualization and then letting the tool generalize edits to other parts of the visualization.

However, multiple modalities also come with their challenges. For example, it is not always easy for users to figure out which modality to use and when. Hence, more research is needed to help

users learn the benefits and disadvantages of each modality so that they make an informed choice.

Static vs. Dynamic UI. One benefit of using natural language commands and dynamic widgets over traditional static UI is the ability to ease the gulf of execution. With static UI, the user has to know how and where to perform the edits, which can be cognitively demanding [15]. Whereas, with NLIs and Dynamic widgets, the users only need to specify their intent. The flip side of this argument is, that despite the learning curve, over time users will learn and get used to the static UIs. However, with dynamic UIs, the interface is constantly changing, which can potentially increase cognitive load for the user, especially in long editing sessions and for users with editing expertise. Some participants from the study did express this concern and suggested having a way of categorizing the widgets predictably. Longer term usability study would provide insights to understand how constantly changing UI affects usability. Unlike traditional interfaces, another limitation of DYNAVIS and other purely NL-based interfaces is that they do not present all possible options to the user at all times. This is a double-edged sword; Due to the ad-hoc nature of NL interfaces, NL commands for adding widgets/editing visualizations can sometimes help users discover previously unknown features (similar to observations in [13]), and other times can shift the onus of discovering the tool’s capabilities onto the user leading to mistrust and distrust [44]. More research is needed to study and find methods to overcome these limitations.

Another interesting future direction is to inspect how we can take advantage of dynamic UI widgets’ low programming requirement to turn end users into “no-code developers” with the ability to customize/DIY their interaction panels to augment static GUI. For example, with dynamic widgets, an end user can construct their own panel that best suits their daily tasks as shortcuts for complex tasks. For example, a user who often works on geographical data analysis can create a custom panel using dynamic widgets specially for map manipulation functions to reduce map editing efforts.

Supporting imperative plotting libraries and lower-level visualization grammars. Dynamic widgets are designed around declarative high-level visualization grammar to enable compositional editing (e.g., VegaLite’s JSON representation for visualization objects). The declarative syntax helps the widgets to be modular and be synthesized and used in any order the user wishes. Despite their advantages, high-level grammars expose fewer options than low-level grammars or imperative libraries for more complex visualization editing tasks (e.g., to make parts of a line dashed while the rest solid, would require visiting lower-level details of how lines are represented). To support editing of visualization in these low-level languages, we envision combining DYNAVIS with bidirectional editing approaches which leverage program analysis and synthesis techniques to propagate surface-level edit requirements to edits over program structures or parameters.

Dynamic widgets for accessibility and other applications. Prior work on dynamically synthesized UIs stemmed from accessibility research, like SUPPLE [25] and SUPPLE++ [26] to accommodate motor and vision capabilities. In the space of dynamic widgets for visualization tasks, there are many possible UIs to perform the same task (e.g., slider vs. number input to control the font size).

Every type of UI has accessibility trade-offs based on the user's needs. In the future, we can imagine a version of DYNAVIS that lets the user provide their interface constraints and preferences, and DYNAVIS automatically synthesizes UI that matches these constraints. Prior work like [33, 53] has accomplished UI synthesis based on examples or demonstrations. A potential research direction would be to add accessibility constraints and exploit the general knowledge of LLMs to create accessible widgets.

While DYNAVIS is designed for visualization editing tasks, we believe dynamic widgets can also benefit general applications that have extensive configuration options (e.g., document processing software, video processing applications). Users can also take advantage of dynamic widgets' low programming requirements in no-code / low-code tools (like Excel / Tableau) with the ability to customize or DIY their interaction panels to augment existing static UI. Investigating how dynamic widgets can be generalized across different application domains is worth studying in the future.

DYNAVIS design opportunities. There are a lot of opportunities to improve DYNAVIS in the future, some of which we highlight here. In the UI_{NL} condition, some participants anticipated repeated edits and copied the NL command, for reuse, before submitting it. In the future versions of DYNAVIS, we can enable users to access the history of NL commands to make re-running NL commands with small edits easier for the user. One of the most requested features from the participants is for more ways to customize and manage a large number of dynamic widgets. This can involve categorizing widgets by topic (like Adobe Photoshop), and collapsing/expanding (sections of) widgets. Another useful feature is to save and revisit certain combinations of widgets. We can also go further by allowing users to export the visualization spec along with the widgets to be shared with other users, similar to Bespoke [53].

7 RELATED WORK

Visualization authoring tools. Modern visualization authoring tools [6, 10, 31, 45, 46] and grammars [5, 47] are built around the grammar of graphics [56] and they greatly reduce the visualization authoring efforts by allowing users to specify high-level visualization intent via mapping of data fields to visual properties. For example, users of Tableau or PowerBI can easily drag data fields and drop into encoding shelves of visual properties to specify the mapping, and users of Vega-Lite can provide mappings concisely as a JSON object. Then, based on high-level specification, these tools automatically provide "smart defaults" to fill low-level visualization properties (e.g., stroke with, spacing of bars) and compiles the visualization spec to low-level visualization grammars like D3 [4] for rendering. While such designs reduce the initial visualization authoring complexity, visualization editing, and refinement remain challenging as the user needs to unbox high-level grammar and navigate through the large space of editing options to perform the edits. DYNAVIS is designed to address the visualization editing challenge, which complements the strengths of existing authoring systems. We envision that DYNAVIS can be combined with existing tools in a way that users start with a high-level specification to describe the visualization intent and then utilize dynamic widgets to perform subsequent edits to refine the chart.

Natural language interfaces for visualization (V-NLIs). Natural language interfaces have been extensively adopted to improve the usability of visualization systems [49]. Even commercial GUI-based tools like Tableau [36], Microsoft Power BI [12], and Google Spreadsheets [21] automatically translate natural language queries to data queries and present query results with visualizations. However, these systems limit natural language interactions to data queries and corresponding standard charts.

The rapid development of Natural Language Processing (NLP) techniques [14, 58] has provided great opportunities to explore a natural language-based interaction for data visualization. There has been active research in adopting Natural Language Interfaces to improve the usability of visualization systems [21, 27, 48, 49, 52, 59]. With the help of advanced NLP-toolkits [2, 3, 9, 32, 35], a surge of visualization-oriented Natural Language Interfaces (V-NLIs) have emerged. V-NLI-based authoring systems accept the user's natural language queries as input and output appropriate visualizations. Researchers have explored multiple techniques ranging from heuristics-based approaches to end-to-end learning approaches.

Heuristic-based approaches explore properties of data in generating a space of potential visualizations [57], ranking these space of visualizations based on quality attributes [34, 37] and presenting them to the user. Further works have considered a task decomposition approach, where the user queries are decomposed into multiple tasks, which are then solved individually and then aggregated to yield the final visualization [16, 38, 55]. Finally, end-to-end learning-based approaches seek to learn mappings from data directly to visualizations [23]. More recently, with the advancements in Large Language Models (LLMs), systems like Lida [22] have found great success in leveraging patterns learned by LLMs from massive language and code datasets to create visualizations from natural language commands. LLMs preclude the requirement of applying heuristics, or training of custom models paired with custom training and data. As an extension, many V-NLI authoring tools also support visualization editing, with natural language as the primary modality.

Customizable and dynamic user interfaces. Prior research in domains such as accessibility and ubiquitous computing has worked on systems that automatically generate UIs. SUPPLE [25] and SUPPLE++ [26] generate custom UIs for users to accommodate their motor and vision capabilities based on user-provided specifications and activity traces. Projects such as UNIFORM [40] and the Personal Universal Controller (PUC) [39] generate custom UIs for appliances such as media consoles and printers that are customized for each individual's preferences and interaction history. Huddle [41] built atop PUC generates UIs to coordinate multiple home electronic appliances. Mavo [54] allows users to create interactive HTML pages without the need for programming by just adding special HTML attributes and also provides different editing widgets based on the type of attributes. DYNAVIS shares these systems' goals of creating specialized UI tailored to individual users' intent.

More recent work on Dynamic interfaces follows a "relaxation" method to create generalized UI widgets. The relaxation method involves creating UI widgets to directly manipulate variables in a function or query. Bespoke [53] synthesizes custom GUIs for

command-line applications by using user demonstrations. They employ rule-based heuristics that infer a semantic type for parameters in bash commands to create a dynamic widget for editing the parameter. Similarly, Heer et al. [28] generates dynamic UI using *query relaxing* that enables the users to generalize their selection. A suite of work named precision interfaces [17, 19, 63] uses SQL queries as a proxy to generate interactive widgets from a sequence of input queries. The latest iteration, NL2Interface [18] generates SQL queries from NL commands and creates a generalized UI to edit the parameters/variables in the SQL query. BOLT [51] and EVIZA [48] generate ambiguity widgets that provide a simple UI for manipulating values for ambiguous inferred variables. e.g., for the NL command “*largest earthquakes in California*”, the threshold for classifying earthquakes as large is ambiguous. However, in both BOLT and EVIZA, natural language commands are restricted by a pre-defined grammar. These tools highlight the importance of complementary GUI tools that accompany NL interfaces. DYNAVIS builds on these systems, and uses an LLM to synthesize dynamic widgets that can enable direct manipulation of Visualization properties. Using LLMs to generate dynamic widgets gives us three distinct advantages:

- (1) By providing an accurate representation of the user’s context to the LLMs, DYNAVIS is less sensitive to errors or ambiguities in natural language commands provided by the user.
- (2) We do not have a fixed set of rules, or heuristics, or rely on query relaxation. Instead of synthesizing a UI that allows users to edit just one variable, LLMs can synthesize widgets that can even capture complex relations like manipulating multiple properties at once.
- (3) Unlike previous systems, we do not restrict the space or the kinds of UI that can be generated. As LLMs become more powerful, this can enable the synthesis of complex interfaces beyond just the traditional HTML Input elements.

Multi-modal user interfaces. Multi-modal interaction techniques have the advantage of letting users better convey their intent in multiple ways reducing the overall effort. Pumice [30] allows users to use natural language to describe programming tasks in end-user development scenarios and then refine intent by providing examples to complement NL’s ambiguous nature. DIY Assistant [24] lets users combine NL and programming specifications to create personal assistants. Lee et al. [29] enables better sense-making with visual query systems with the help of sketching. ShapeSearch [50] lets users use query shapes using both NL and regular expressions — greatly improving the expressiveness of shape search queries. Tools like PanaromicData [61], and Vizdom [20] allow users to use pen and touch to directly perform data aggregation and analytics respectively on a digital whiteboard. DYNAVIS builds on the idea of enabling multiple modalities of interaction. DYNAVIS leverages both NL-based interaction to reduce the gulf of execution and UI-based interaction to enhance interactivity. In the future, DYNAVIS can further combine pen-and-touch for direct control of visual elements on canvas as well as sketching to demonstrate editing effects.

8 CONCLUSION

In this paper, we introduce DYNAVIS, which blends natural language and dynamically synthesized UI widgets to ease the gulf

of execution and enhance interactivity. Given a visualization edit command or a widget creation command, DYNAVIS synthesizes a UI widget that the user can interact with to perform visualization edits. Our study with 24 participants shows that participants preferred DYNAVIS over the NLI-only interface citing ease of further edits and editing confidence due to immediate visual feedback.

ACKNOWLEDGMENTS

This material is based upon work supported by the NSF under Grant No. CCF-2123965. This work was mostly performed during an internship at Microsoft, and we thank the VIDA and the DL team at Microsoft Research for their support and feedback.

REFERENCES

- [1] 2020. OpenAI API. <https://openai.com/blog/openai-api>. Accessed: 2023-9-13.
- [2] 2023. Apache OpenNLP. <https://opennlp.apache.org/>. Accessed: 2023-9-13.
- [3] 2023. Cloud Natural Language. <https://cloud.google.com/natural-language>. Accessed: 2023-9-13.
- [4] 2023. D3 by Observable. <https://d3js.org/>. Accessed: 2023-9-14.
- [5] 2023. ggplot2. <https://ggplot2.tidyverse.org/>. Accessed: 2023-9-14.
- [6] 2023. Microsoft PowerBI. <https://powerbi.microsoft.com/en-us/>. Accessed: 2023-9-14.
- [7] 2023. npm: html-react-parser. <https://www.npmjs.com/package/html-react-parser>. Accessed: 2023-12-6.
- [8] 2023. pandas documentation — pandas 2.1.0 documentation. <https://pandas.pydata.org/docs/index.html>. Accessed: 2023-9-14.
- [9] 2023. spaCy - Industrial-strength Natural Language Processing in Python. <https://spacy.io/>. Accessed: 2023-9-13.
- [10] 2023. Tableau: Business Intelligence and Analytics Software. <https://www.tableau.com/>. Accessed: 2023-9-14.
- [11] 2023. The Two UX Gulfs: Evaluation and Execution. <https://www.nngroup.com/articles/two-ux-gulfs-evaluation-execution/>. Accessed: 2023-9-14.
- [12] 2023. Use natural language to explore data with Power BI Q&A - Power BI. <https://learn.microsoft.com/en-us/power-bi/natural-language/q-and-a-intro>. Accessed: 2023-9-13.
- [13] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proc. ACM Program. Lang.* 7, OOPSLA1, Article 78 (apr 2023), 27 pages. <https://doi.org/10.1145/3586030>
- [14] Yonatan Belinkov and James Glass. 2019. Analysis methods in neural language processing: A survey. *Transactions of the Association for Computational Linguistics* 7 (2019), 49–72.
- [15] Raluca Budi. 2014. Memory Recognition and Recall in User Interfaces. <https://www.nngroup.com/articles/recognition-and-recall/>. Accessed: 2023-9-12.
- [16] Qiaochu Chen, Shankara Pailoor, Celeste Barnaby, Abby Criswell, Chenglong Wang, Greg Durrett, and İşıl Dillig. 2022. Type-directed synthesis of visualizations from natural language queries. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 532–559.
- [17] Yiru Chen. 2020. Monte Carlo Tree Search for Generating Interactive Data Analysis Interfaces. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data* (Portland, OR, USA) (SIGMOD ’20). Association for Computing Machinery, New York, NY, USA, 2837–2839. <https://doi.org/10.1145/3318464.3384404>
- [18] Yiru Chen, Ryan Li, Austin Mac, Tianbao Xie, Tao Yu, and Eugene Wu. 2022. NL2INTERFACE: Interactive Visualization Interface Generation from Natural Language Queries. *ArXiv abs/2209.08834* (2022). <https://api.semanticscholar.org/CorpusID:252367337>
- [19] Yiru Chen and Eugene Wu. 2022. PI2: End-to-end Interactive Visualization Interface Generation from Queries. In *Proceedings of the 2022 International Conference on Management of Data* (Philadelphia, PA, USA) (SIGMOD ’22). Association for Computing Machinery, New York, NY, USA, 1711–1725. <https://doi.org/10.1145/3514221.3526166>
- [20] Andrew Crotty, Alex Galakatos, Emanuel Zraggen, Carsten Binnig, and Tim Kraska. 2015. Vizdom: interactive analytics through pen and touch. *Proceedings of the VLDB Endowment* 8, 12 (2015), 2024–2027.
- [21] Kedar Dhamdhere, Kevin S. McCurley, Ralfi Nahmias, Mukund Sundararajan, and Qiqi Yan. 2017. Analyza: Exploring Data with Conversation. In *Proceedings of the 22nd International Conference on Intelligent User Interfaces* (Limassol, Cyprus) (IUI ’17). Association for Computing Machinery, New York, NY, USA, 493–504. <https://doi.org/10.1145/3025171.3025227>
- [22] Victor Dibia. 2023. LIDA: A Tool for Automatic Generation of Grammar-Agnostic Visualizations and Infographics using Large Language Models. (6 March 2023).

- arXiv:2303.02927 [cs.AI]
- [23] Victor Dibia and Çağatay Demiralp. 2019. Data2vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks. *IEEE computer graphics and applications* 39, 5 (2019), 33–46.
 - [24] Michael H. Fischer, Giovanni Campagna, Euirim Choi, and Monica S. Lam. 2021. DIY assistant: a multi-modal end-user programmable virtual assistant. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (Virtual, Canada) (PLDI 2021)*. Association for Computing Machinery, New York, NY, USA, 312–327. <https://doi.org/10.1145/3453483.3454046>
 - [25] Krzysztof Gajos and Daniel S. Weld. 2004. SUPPLE: automatically generating user interfaces. In *Proceedings of the 9th International Conference on Intelligent User Interfaces (Funchal, Madeira, Portugal) (IUI '04)*. Association for Computing Machinery, New York, NY, USA, 93–100. <https://doi.org/10.1145/964442.964461>
 - [26] Krzysztof Z. Gajos, Jacob O. Wobbrock, and Daniel S. Weld. 2007. Automatically generating user interfaces adapted to users' motor and vision capabilities. In *Proceedings of the 20th Annual ACM Symposium on User Interface Software and Technology (Newport, Rhode Island, USA) (UIST '07)*. Association for Computing Machinery, New York, NY, USA, 231–240. <https://doi.org/10.1145/1294211.1294253>
 - [27] Tong Gao, Mira Dontcheva, Eytan Adar, Zhicheng Liu, and Karrie Karahalios. 2015. DataTone: Managing Ambiguity in Natural Language Interfaces for Data Visualization. In *Proceedings of the 28th Annual ACM Symposium on User Interface Software & Technology (Charlotte, NC, USA) (UIST '15)*. Association for Computing Machinery, New York, NY, USA, 489–500. <https://doi.org/10.1145/2807442.2807478>
 - [28] Jeffrey Heer, Maneesh Agrawala, and Wesley Willett. 2008. Generalized selection via interactive query relaxation. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (Florence, Italy) (CHI '08)*. Association for Computing Machinery, New York, NY, USA, 959–968. <https://doi.org/10.1145/1357054.1357203>
 - [29] Doris Jung-Lin Lee, John Lee, Tarique Siddiqui, Jaewoo Kim, Karrie Karahalios, and Aditya Parameswaran. 2019. You can't always sketch what you want: Understanding sensemaking in visual query systems. *IEEE transactions on visualization and computer graphics* 26, 1 (2019), 1267–1277.
 - [30] Toby Jia-Jun Li, Marissa Radensky, Justin Jia, Kirielle Singarajah, Tom M. Mitchell, and Brad A. Myers. 2019. PUMICE: A Multi-Modal Agent that Learns Concepts and Conditionals from Natural Language and Demonstrations. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (New Orleans, LA, USA) (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 577–589. <https://doi.org/10.1145/3332165.3347899>
 - [31] Zhicheng Liu, John Thompson, Alan Wilson, Mira Dontcheva, James Delorey, Sam Grigg, Bernard Kerr, and John Stasko. 2018. Data Illustrator: Augmenting Vector Design Tools with Lazy Data Binding for Expressive Visualization Authoring. In *Proceedings of the 2018 CHI Conference on Human Factors in Computing Systems (Montreal QC, Canada.) (CHI '18)*. Association for Computing Machinery, New York, NY, USA, 1–13. <https://doi.org/10.1145/3173574.3173697>
 - [32] Edward Loper and Steven Bird. 2002. Nltk: The natural language toolkit. *arXiv preprint cs/0205028* (2002).
 - [33] Dylan Lukes, John Sarracino, Cora Coleman, Hila Peleg, Sorin Lerner, and Nadia Polikarpova. 2021. Synthesis of web layouts from examples. In *Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (Athens, Greece) (ESEC/FSE 2021)*. Association for Computing Machinery, New York, NY, USA, 651–663. <https://doi.org/10.1145/3468264.3468533>
 - [34] Yuyu Luo, Xuedi Qin, Nan Tang, Guoliang Li, and Xinran Wang. 2018. DeepEye: Creating Good Data Visualizations by Keyword Search. In *Proceedings of the 2018 International Conference on Management of Data (Houston, TX, USA) (SIGMOD '18)*. Association for Computing Machinery, New York, NY, USA, 1733–1736. <https://doi.org/10.1145/3183713.3193545>
 - [35] Christopher Manning, Mihai Surdeanu, John Bauer, Jenny Finkel, Steven Bethard, and David McClosky. 2014. The Stanford CoreNLP Natural Language Processing Toolkit. In *Proceedings of 52nd Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, Kalina Bontcheva and Jingbo Zhu (Eds.). Association for Computational Linguistics, Baltimore, Maryland, 55–60. <https://doi.org/10.3115/v1/P14-5010>
 - [36] Ruhaab Markas. 2018. Ask Data: Simplifying analytics with natural language. <https://www.tableau.com/blog/ask-data-simplifying-analytics-natural-language-98655>. Accessed: 2023-9-13.
 - [37] Dominik Moritz, Chenglong Wang, Greg L. Nelson, Halden Lin, Adam M Smith, Bill Howe, and Jeffrey Heer. 2018. Formalizing visualization design knowledge as constraints: Actionable and extensible models in draco. *IEEE transactions on visualization and computer graphics* 25, 1 (2018), 438–448.
 - [38] Arpit Narechania, Arjun Srinivasan, and John Stasko. 2020. NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries. *IEEE Transactions on Visualization and Computer Graphics* 27, 2 (2020), 369–379.
 - [39] Jeffrey Nichols, Brad A. Myers, Michael Higgins, Joseph Hughes, Thomas K. Harris, Roni Rosenfeld, and Kevin Litwack. 2003. Personal universal controllers: controlling complex appliances with GUIs and speech. In *CHI '03 Extended Abstracts on Human Factors in Computing Systems* (Ft. Lauderdale, Florida, USA) (CHI EA '03). Association for Computing Machinery, New York, NY, USA, 624–625. <https://doi.org/10.1145/765891.765896>
 - [40] Jeffrey Nichols, Brad A. Myers, and Brandon Rothrock. 2006. UNIFORM: automatically generating consistent remote control user interfaces. In *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems (Montréal, Québec, Canada) (CHI '06)*. Association for Computing Machinery, New York, NY, USA, 611–620. <https://doi.org/10.1145/1124772.1124865>
 - [41] Jeffrey Nichols, Brandon Rothrock, Duen Horng Chau, and Brad A. Myers. 2006. Huddle: automatically generating interfaces for systems of multiple connected appliances. In *Proceedings of the 19th Annual ACM Symposium on User Interface Software and Technology (Montreux, Switzerland) (UIST '06)*. Association for Computing Machinery, New York, NY, USA, 279–288. <https://doi.org/10.1145/1166253.1166298>
 - [42] Jakob Nielsen. 2020. 10 Usability Heuristics for User Interface Design. <https://www.nngroup.com/articles/ten-usability-heuristics/>. Accessed: 2023-9-13.
 - [43] Donald Norman. 1986. User centered system design. *New perspectives on human-computer interaction* (1986).
 - [44] Raja Parasuraman and Victor Riley. 1997. Humans and automation: Use, misuse, disuse, abuse. *Human factors* 39, 2 (1997), 230–253.
 - [45] Donghao Ren, Bongshin Lee, and Matthew Brehmer. 2018. Chartulator: Interactive construction of bespoke chart layouts. *IEEE transactions on visualization and computer graphics* 25, 1 (2018), 789–799.
 - [46] Arvind Satyanarayan and Jeffrey Heer. 2014. Lyra: An Interactive Visualization Design Environment. *Computer Graphics Forum* 33, 3 (2014), 351–360. <https://doi.org/10.1111/cgf.12391> arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1111/cgf.12391
 - [47] Arvind Satyanarayan, Dominik Moritz, Kanit Wongsuphasawat, and Jeffrey Heer. 2016. Vega-lite: A grammar of interactive graphics. *IEEE transactions on visualization and computer graphics* 23, 1 (2016), 341–350.
 - [48] Vidya Setlur, Sarah E. Battersby, Melanie Tory, Rich Gossweiler, and Angel X. Chang. 2016. Eviza: A Natural Language Interface for Visual Analysis. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (Tokyo, Japan) (UIST '16)*. Association for Computing Machinery, New York, NY, USA, 365–377. <https://doi.org/10.1145/2984511.2984588>
 - [49] Leixian Shen, Enya Shen, Yuyu Luo, Xiaocong Yang, Xuming Hu, Xiongshuai Zhang, Zhiwei Tai, and Jianmin Wang. 2022. Towards natural language interfaces for data visualization: A survey. *IEEE transactions on visualization and computer graphics* (2022).
 - [50] Tarique Siddiqui, Paul Luh, Zesheng Wang, Karrie Karahalios, and Aditya Parameswaran. 2020. ShapeSearch: A Flexible and Efficient System for Shape-based Exploration of Trendlines. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (Portland, OR, USA) (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 51–65. <https://doi.org/10.1145/3318464.3389722>
 - [51] Arjun Srinivasan and Vidya Setlur. 2023. BOLT: A Natural Language Interface for Dashboard Authoring. (2023).
 - [52] Arjun Srinivasan and John T. Stasko. 2017. Natural Language Interfaces for Data Analysis with Visualization: Considering What Has and Could Be Asked. In *EuroVis 2017 - Short Papers*, Barbora Kozlikova, Tobias Schreck, and Thomas Wischgoll (Eds.). The Eurographics Association, 55–59. <https://doi.org/10.2312/eurovisshort.20171133>
 - [53] Priyan Vaithilingam and Philip J. Guo. 2019. Bespoke: Interactively Synthesizing Custom GUIs from Command-Line Applications By Demonstration. In *Proceedings of the 32nd Annual ACM Symposium on User Interface Software and Technology (New Orleans, LA, USA) (UIST '19)*. Association for Computing Machinery, New York, NY, USA, 563–576. <https://doi.org/10.1145/3332165.3347944>
 - [54] Lea Verou, Amy X. Zhang, and David R. Karger. 2016. Mavo: Creating Interactive Data-Driven Web Applications by Authoring HTML. In *Proceedings of the 29th Annual Symposium on User Interface Software and Technology (Tokyo, Japan) (UIST '16)*. Association for Computing Machinery, New York, NY, USA, 483–496. <https://doi.org/10.1145/2984511.2984551>
 - [55] Yun Wang, Zhitao Hou, Leixian Shen, Tongshuang Wu, Jiaqi Wang, He Huang, Haidong Zhang, and Dongmei Zhang. 2022. Towards natural language-based visualization authoring. *IEEE Transactions on Visualization and Computer Graphics* 29, 1 (2022), 1222–1232.
 - [56] Leland Wilkinson. 2005. *The Grammar of Graphics, Second Edition*. Springer.
 - [57] Kanit Wongsuphasawat, Zening Qu, Dominik Moritz, Riley Chang, Felix Ouk, Anushka Anand, Jock Mackinlay, Bill Howe, and Jeffrey Heer. 2017. Voyager 2: Augmenting Visual Analysis with Partial View Specifications. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems (Denver, Colorado, USA) (CHI '17)*. Association for Computing Machinery, New York, NY, USA, 2648–2659. <https://doi.org/10.1145/3025453.3025768>
 - [58] Tom Young, Devamanyu Hazarika, Soujanya Poria, and Erik Cambria. 2018. Recent trends in deep learning based natural language processing. *IEEE Computational intelligence magazine* 13, 3 (2018), 55–75.

- [59] Bowen Yu and Cláudio T Silva. 2019. FlowSense: A natural language interface for visual data exploration within a dataflow system. *IEEE transactions on visualization and computer graphics* 26, 1 (2019), 1–11.
- [60] JD Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*. 1–21.
- [61] Emanuel Zgraggen, Robert Zeleznik, and Steven M Drucker. 2014. Panoramicdata: Data analysis through pen & touch. *IEEE transactions on visualization and computer graphics* 20, 12 (2014), 2112–2121.
- [62] Dan Zhang, Yoshihiko Suhara, Jinfeng Li, Madelon Hulsebos, Çağatay Demiralp, and Wang-Chiew Tan. 2019. Sato: Contextual semantic type detection in tables. *arXiv preprint arXiv:1911.06311* (2019).
- [63] Haoci Zhang, Viraj Raj, Thibault Sellam, and Eugene Wu. 2018. Precision interfaces for different modalities. In *Proceedings of the 2018 International Conference on Management of Data*. 1777–1780.